

GRA-Unified-Swarm-Lean: двухполюсная архитектура генерации гипотез и формальной верификации

GRA-Unified-Swarm-Lean:
A Two-Pole Architecture of Hypothesis Generation and Formal Verification

Олег Бит / Oleg Bit
Independent Researcher / Engineer
2026

3 октября 2026 г.

Аннотация

RU: Представлена архитектура **GRA-Unified-Swarm-Lean**, объединяющая два противоположных полюса интеллектуальной системы: **GRA-обнуление** + **Unified Swarm** как генератор и разрушитель гипотез, и **Lean 4** как формальный судья. Ключевая идея — не заставлять один полюс выполнять работу другого, а построить замкнутый **feedback loop**: GRA порождает кандидатов, Lean возвращает структурные ошибки, GRA использует эти ошибки для следующего поколения гипотез. Вводится пять ролей агентов (Generator, Contradiction Hunter, Nullifier, Formalizer, Verifier). Lean трактуется не только как судья, но и как **сенсор формальной обратной связи**. Показано, как из множества ошибок Lean извлекается **мета-гипотеза** — структурная информация, недоступная одиночному self-critique. Описывается воспроизводимый in silico benchmark, моделирующий эту архитектуру с синтетическим verifier, и честно обозначается граница: настоящий Lean-контур требует отдельного эксперимента.

EN: We present the **GRA-Unified-Swarm-Lean** architecture, which combines two opposite poles of an intelligent system: **GRA-nullification** + **Unified Swarm** as generator and destroyer of hypotheses, and **Lean 4** as formal judge. The key idea is not to force one pole to do the other's job, but to build a closed **feedback loop**: GRA produces candidates, Lean returns structural errors, GRA uses these errors for the next generation of hypotheses. Five agent roles are introduced (Generator, Contradiction Hunter, Nullifier, Formalizer, Verifier). Lean is treated not only as a judge but as a **formal feedback sensor**. We show how **meta-hypotheses** — structural information unavailable to single self-critique — are extracted from multiple Lean errors. A reproducible in silico benchmark modeling this architecture with a synthetic verifier is described, and the boundary is honestly stated: a real Lean loop requires a separate experiment.

1 Введение: два полюса, не конкуренция / Introduction: Two Poles, Not Competition

RU: Современная формальная верификация и современные генеративные системы решают принципиально разные задачи. **Lean 4** — чрезвычайно строгий формальный контур: доказательство принимается только тогда, когда оно проходит проверку доверенным ядром. Это делает Lean мощным инструментом верификации, но само пространство допустимых результатов определяется формальным языком, аксиомами, типами и поставленной задачей.

EN: Modern formal verification and modern generative systems solve fundamentally different problems. **Lean 4** is an extremely strict formal loop: a proof is accepted only when it passes the trusted kernel check. This makes Lean a powerful verification tool, but the space of admissible results is determined by the formal language, axioms, types, and the task at hand.

RU: GRA-Unified Swarm предлагает противоположное направление: не начинать с заданного пространства допустимых доказательств, а **постоянно расширять пространство гипотез**, атаковать существующие гипотезы противоречиями, разрушать неустойчивые структуры и строить новые кандидаты.

EN: GRA-Unified Swarm proposes the opposite direction: not to start from a pre-given space of admissible proofs, but to **continuously expand the hypothesis space**, attack existing hypotheses with contradictions, destroy unstable structures, and build new candidates.

RU: Возникает не конкуренция двух подходов, а двухполюсная архитектура:

EN: What emerges is not a competition between two approaches, but a two-pole architecture:

GRA = generator and destroyer of hypotheses

Lean 4 = formal judge

RU: Между полюсами должен существовать **feedback loop**:

EN: Between the poles there must exist a **feedback loop**:

GRA → Hypothesis → Lean → Error → GRA → New Hypothesis

RU: Ключевое требование: **не соединять полюса напрямую**. GRA не имеет права объявлять «доказано». Lean не обязан ничего генерировать. Их роли строго разделены.

EN: The key requirement: **do not connect the poles directly**. GRA has no right to declare “proved.” Lean is not obliged to generate anything. Their roles are strictly separated.

2 Две оси: H и V / Two Axes: H and V

RU: Архитектурно Lean оптимизирован под **формальную допустимость**, GRA — под расширение пространства гипотез. Введём два независимых измерения:

EN: Architecturally, Lean is optimized for **formal admissibility**, GRA for expanding the hypothesis space. Let us introduce two independent dimensions:

H = generative breadth, V = formal validity.

Lean: $H_{\text{task}} \rightarrow V_{\text{max}}$, GRA-Swarm: $H \rightarrow H_{\text{expanded}} \rightarrow V$.

RU: Это не «Lean vs GRA», а:

EN: This is not “Lean vs GRA” but:

GRA expands the hypothesis space

Lean contracts the accepted space

3 Пространство гипотез / The Hypothesis Space

RU: Пусть $\mathcal{H} = \{h_1, h_2, \dots, h_N\}$ — пространство возможных гипотез. В модельном масштабе:

EN: Let $\mathcal{H} = \{h_1, h_2, \dots, h_N\}$ be the space of possible hypotheses. At the model scale:

$$|\mathcal{H}| \approx 10^{150}, \quad \log_{10} |\mathcal{H}| = 150.$$

RU: Это не утверждение, что GRA физически проверяет 10^{150} гипотез. Это модель потенциального пространства поиска. Если реально исследовано N_{eval} кандидатов:

EN: This is not a claim that GRA physically verifies 10^{150} hypotheses. It is a model of the potential search space. If N_{eval} candidates are actually explored:

$$\kappa = \frac{N_{\text{eval}}}{10^{150}}.$$

RU: При $N_{\text{eval}} = 10^5$: $\kappa = 10^{-145}$. GRA не перебирает — она **навигирует**. Lean проверяет только выбранные кандидаты: $N_{\text{Lean}} \ll N$. Каждый цикл может создавать новые области: $\mathcal{H}_0 \rightarrow \mathcal{H}_1 \rightarrow \mathcal{H}_2$.

EN: With $N_{\text{eval}} = 10^5$: $\kappa = 10^{-145}$. GRA does not enumerate — it **navigates**. Lean verifies only selected candidates: $N_{\text{Lean}} \ll N$. Each cycle may create new regions: $\mathcal{H}_0 \rightarrow \mathcal{H}_1 \rightarrow \mathcal{H}_2$.

4 Пять ролей Unified Swarm / Five Roles of the Unified Swarm

RU: Для GRA-Unified Swarm вводится минимум пять агентов, каждый со специализированной функцией.

EN: For GRA-Unified Swarm, at least five agents are introduced, each with a specialized function.

4.1 Agent 1 — Generator / Агент 1 — Генератор

$$G(H_t) \rightarrow \{H_1, H_2, \dots, H_N\}.$$

RU: Пример гипотез о теореме T : « T следует из B », « T следует из C », « T требует дополнительного условия D », « T ложна», « T верна только для конечных n ».

EN: Example hypotheses about a theorem T : “ T follows from B ,” “ T follows from C ,” “ T requires an additional condition D ,” “ T is false,” “ T holds only for finite n .”

4.2 Agent 2 — Contradiction Hunter / Агент 2 — Охотник за противоречиями

$$C(H_i) \in \{0, 1\}.$$

RU: Ищет контрпример, противоречие с аксиомами, нарушение ограничения, неправильный тип, крайний случай, скрытое предположение. **Ключевое:** GRA не просто удаляет H_1 — она спрашивает: *почему H_1 сломалась?*

EN: Searches for a counterexample, contradiction with axioms, constraint violation, wrong type, edge case, hidden assumption. **Key:** GRA does not simply delete H_1 — it asks: *why did H_1 break?*

4.3 Agent 3 — GRA Nullifier / Агент 3 — Обнулитель

RU: Центральная часть концепции. Пусть $H_1 = A + B + C + D$, и противоречие связано только с C . Тогда:

EN: The central part of the concept. Let $H_1 = A + B + C + D$, and the contradiction is associated only with C . Then:

$$\mathcal{N}(H_1) = A + B + \emptyset + D, \quad H'_1 = A + B + C' + D.$$

$$\boxed{H \rightarrow \text{Contradiction} \rightarrow \text{Nullification} \rightarrow \text{Structural Revision}}$$

RU: Это существенно отличается от обычного **wrong** -> **delete**.

EN: This differs fundamentally from the usual **wrong** -> **delete**.

4.4 Agent 4 — Lean Formalizer / Агент 4 — Формализатор

RU: После ревизии GRA выдаёт гипотезу, Formalizer переводит её в Lean:

EN: After revision GRA produces a hypothesis; the Formalizer translates it into Lean:

```
theorem candidate
  (n : Nat)
  (h : Even n) :
  P n := by
  ...
```

RU: Важно: **GRA** не должна писать Lean-код идеально с первого раза. Пусть напишет неправильный код — это даже полезно. Lean ответит `unsolved goals`, и это становится новым GRA-сигналом.

EN: Important: **GRA** must not write Lean code perfectly on the first try. Let it write incorrect code — this is even useful. Lean will respond `unsolved goals`, and this becomes a new GRA signal.

4.5 Agent 5 — Lean Verifier / Агент 5 — Верификатор

$$\text{Verify}(H) = \begin{cases} 1, & \text{Lean accepts} \\ 0, & \text{Lean rejects} \end{cases}$$

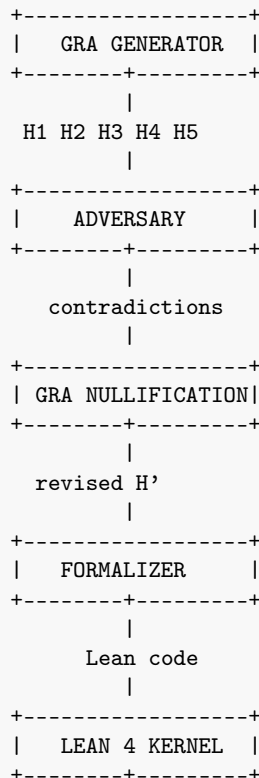
RU: GRA может сказать «это очень красивая гипотеза». Lean отвечает: «Мне всё равно. Покажи формальное доказательство». Именно это делает систему интересной.

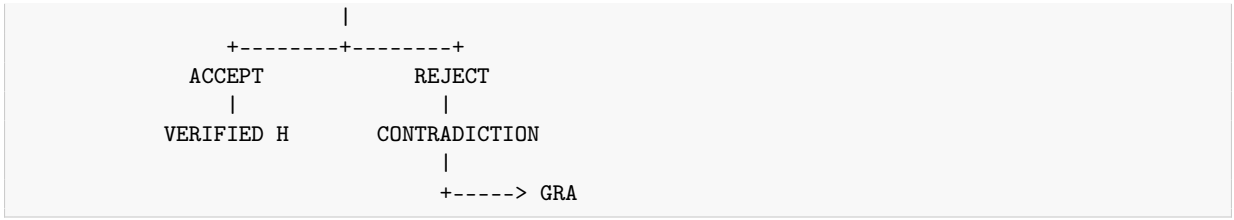
EN: GRA may say “this is a very beautiful hypothesis.” Lean answers: “I don’t care. Show me a formal proof.” This is precisely what makes the system interesting.

5 Полный цикл / Full Cycle

RU: Схема взаимодействия ролей:

EN: The interaction scheme of the roles:





6 Lean как формальный сенсор / Lean as a Formal Sensor

RU: Это самая сильная идея архитектуры. Не «GRA генерирует, Lean проверяет» — это только половина. Правильнее:

EN: This is the strongest idea of the architecture. Not “GRA generates, Lean verifies” — that is only half. More correctly:

Lean = formal feedback sensor

RU: Lean сообщает не бинарный результат, а структурный сигнал:

EN: Lean reports not a binary result but a structural signal:

- **RU:** неверный тип / **EN:** wrong type,
- **RU:** отсутствует предположение / **EN:** missing assumption,
- **RU:** невозможно вывести утверждение / **EN:** cannot derive the statement,
- **RU:** не закрыта цель / **EN:** unsolved goal,
- contradiction,
- proof accepted.

RU: GRA использует эти сигналы для следующей генерации:

EN: GRA uses these signals for the next generation:

$$H_t \xrightarrow{\text{GRA}} P_t \xrightarrow{\text{Lean}} E_t \xrightarrow{\text{GRA}} H_{t+1}.$$

7 Пример структурной ревизии / Example of Structural Revision

RU: Допустим, GRA придумала теорему:

EN: Suppose GRA came up with a theorem:

```
theorem T (n : Nat) :
  n^2 + n + 1 = (n+1)^2 := by
  ring
```

RU: Lean сообщает `ring failed`. GRA получает не просто `false`, а структурный feedback:

EN: Lean reports `ring failed`. GRA receives not just `false` but structural feedback:

```
LHS = n^2 + n + 1
RHS = n^2 + 2n + 1
missing n
```

RU: Тогда GRA строит:

EN: Then GRA builds:

$$H_{t+1} = R(H_t, E_{\text{Lean}}).$$

RU: Например, ревизия:

EN: For example, revision:

```
theorem T (n : Nat) :
  n^2 + 2*n + 1 = (n+1)^2 := by
  ring
```

RU: Lean отвечает АСЦЕПТ. Это и есть настоящий **GRA** $\rightarrow$ **Lean** $\rightarrow$ **GRA loop**.

EN: Lean responds ACCEPT. This is the genuine **GRA** $\rightarrow$ **Lean** $\rightarrow$ **GRA loop**.

8 Мета-гипотеза из множества ошибок / Meta-Hypothesis from Multiple Errors

RU: Не один GRA-агент должен получать ошибку Lean. Пусть есть роли:

EN: Not one GRA agent must receive the Lean error. Let there be roles:

Agent	Role
G1	Generator
G2	Mathematician
G3	Counterexample Hunter
G4	Formalizer
G5	Lean Debugger
G6	Adversary
G7	Synthesizer

RU: Каждый получает одну и ту же проблему, но **не одну и ту же задачу**. Пусть после цикла:

EN: Each receives the same problem, but **not the same task**. Suppose after a cycle:

```
H1 -> Lean FAIL
H2 -> Lean FAIL
H3 -> Lean ACCEPT
H4 -> Lean FAIL
H5 -> Lean FAIL
```

RU: GRA не должна просто выбрать H3. Она анализирует E_1, E_2, E_4, E_5 . Возможно, четыре ошибки показывают общий паттерн:

EN: GRA must not simply choose H3. It analyzes E_1, E_2, E_4, E_5 . Perhaps four errors show a common pattern:

```
H1: missing positivity assumption
H2: missing positivity assumption
H4: missing positivity assumption
H5: missing positivity assumption
```

RU: Тогда появляется мета-гипотеза:

EN: Then a meta-hypothesis appears:

$$T \text{ requires } x > 0$$

RU: Это уже не просто исправление одного proof. Это **извлечение структурной информации из множества провалов**. Именно здесь GRA становится интереснее обычного self-critique.

EN: This is no longer a fix of a single proof. It is **extraction of structural information from multiple failures**. This is where GRA becomes more interesting than ordinary self-critique.

Self-critique:

$$H \rightarrow \text{Critic} \rightarrow H'$$

GRA-Swarm:

$$\{H_1, \dots, H_N\} \rightarrow \{E_1, \dots, E_N\} \rightarrow \text{common structure} \rightarrow H_{\text{meta}}.$$

$$\text{error} \rightarrow \text{error structure} \rightarrow \text{meta-hypothesis}$$

9 Парето-фронт гипотез / Pareto Front of Hypotheses

RU: Вместо одной гипотезы можно оценивать вектор:

EN: Instead of a single hypothesis, one may evaluate a vector:

$$J(H) = (\text{proof difficulty, complexity, novelty, verification, runtime}).$$

RU: Пример:

EN: Example:

Hypothesis	Complexity	Novelty	Proof diff.	Lean
H1	0.2	0.3	0.2	FAIL
H2	0.4	0.8	0.6	FAIL
H3	0.6	0.9	0.5	PASS
H4	0.3	0.7	0.3	FAIL
H5	0.8	0.95	0.7	PASS

RU: Тогда нельзя просто сказать $H_3 = \text{best}$. Можно получить:

EN: Then one cannot simply say $H_3 = \text{best}$. One may obtain:

$$\mathcal{P}_H = \text{Pareto}(H_1, \dots, H_N).$$

RU: И уже внутри Парето-фронта — пересечение с формально допустимым подмножеством:

EN: And within the Pareto front — the intersection with the formally admissible subset:

$$\mathcal{P}_{\text{final}} = \mathcal{P}_H \cap \mathcal{H}_{\text{Lean}}.$$

10 Центральные формулы архитектуры / Central Formulas

RU: Центральная формула генерации следующего поколения гипотез:

EN: The central formula for generating the next generation of hypotheses:

$$\boxed{\mathcal{H}_{t+1} = \mathcal{G}[\mathcal{H}_t, \mathcal{C}_t, \mathcal{E}_t^{\text{Lean}}, \mathcal{M}_t]}$$

RU: где:

- $\mathcal{H}_t$ — текущие гипотезы / current hypotheses,
- $\mathcal{C}_t$ — обнаруженные противоречия / detected contradictions,
- $\mathcal{E}_t^{\text{Lean}}$ — ошибки формальной проверки / formal verification errors,
- $\mathcal{M}_t$ — мета-информация роя / meta-information of the swarm.

RU: Верификация:

EN: Verification:

$$\boxed{\mathcal{V}_t = \text{Lean}(\text{Formalize}(\mathcal{H}_t))}$$

RU: Итоговое множество верифицированных гипотез:

EN: The final set of verified hypotheses:

$$\boxed{\mathcal{H}_{\text{verified}} = \{h \in \mathcal{H} : V(h) = 1\}}$$

RU: Полная схема:

EN: The full schema:

$$\mathcal{H} \xrightarrow{\text{GRA}} \mathcal{H}' \xrightarrow{\text{Contr.}} \mathcal{H}'' \xrightarrow{\text{Nullif.}} \mathcal{H}''' \xrightarrow{\text{Lean}} \mathcal{H}_{\text{verified}}$$

11 Резонанс двух полюсов / Resonance of the Two Poles

RU: Два полюса — не противоположности, а **дополнения**:

EN: The two poles are not opposites but **complements**:

GRA expands the hypothesis space

Lean contracts the accepted space

RU: Unified Swarm находится между ними:

EN: Unified Swarm resides between them:

explore broadly $\longleftrightarrow$ verify strictly

RU: Только гипотезы, прошедшие Lean, попадают в $\mathcal{H}_{\text{verified}}$. GRA не имеет права объявлять «доказано».

EN: Only hypotheses that pass Lean enter $\mathcal{H}_{\text{verified}}$. GRA has no right to declare “proved.”

12 Практическая реализация / Practical Implementation

12.1 Уровень 1 — Python / Level 1 — Python

RU: Сначала без настоящего Lean. Быстрая проверка алгоритма:

EN: At first without real Lean. Fast algorithm check:

```

GRA Generator
  |
Synthetic contradiction
  |
Nullification
  |
Revision
  |
Synthetic verifier
    
```

12.2 Уровень 2 — GRA + Lean 4 / Level 2 — GRA + Lean 4

RU: Затем реальный Lean:

EN: Then real Lean:

```

Python GRA
  |
hypothesis
  |
LLM / Formalizer
  |
candidate.lean
  |
Lean 4
  |
result
    
```

RU: Python создаёт файлы и получает SUCCESS или ERROR:

EN: Python creates files and receives SUCCESS or ERROR:

```
lake env lean candidate_001.lean
```

12.3 Уровень 3 — Настоящий Unified GRA / Level 3 — Real Unified GRA

RU: Структура проекта:

EN: Project structure:

```
GRA-Lean/
|
+-- gra/
|   +-- generator.py
|   +-- contradiction.py
|   +-- nullification.py
|   +-- revision.py
|   +-- pareto.py
|   +-- swarm.py
|
+-- lean/
|   +-- Main.lean
|   +-- Candidate.lean
|   +-- Verified.lean
|
+-- bridge/
|   +-- formalizer.py
|   +-- lean_runner.py
|   +-- parser.py
|
+-- experiments/
|   +-- benchmark.py
|   +-- monte_carlo.py
|   +-- results/
|
+-- README.md
```

12.4 Интерфейс GRA $\leftrightarrow$ Lean

RU: GRA отправляет JSON:

EN: GRA sends JSON:

```
{
  "id": "H_00421",
  "hypothesis": "...",
  "formal_statement": "...",
  "priority": 0.87
}
```

RU: Lean возвращает:

EN: Lean returns:

```
{
  "id": "H_00421",
  "status": "FAIL",
  "errors": [
    { "type": "unsolved_goal", "message": "..."}
  ]
}
```

RU: Тогда GRA получает $\mathcal{E}_{\text{Lean}}$.

EN: Then GRA receives $\mathcal{E}_{\text{Lean}}$.

13 In silico верификация / In Silico Verification

13.1 Протокол / Protocol

RU: Сравниваются пять методов на синтетической задаче с коррелированными ошибками $\rho \in [0, 1]$:

EN: Five methods are compared on a synthetic task with correlated errors $\rho \in [0, 1]$:

1. **Static** — одиночный агент / single agent,
2. **Debate** — multi-agent debate,
3. **Self-Critique** — генерация + критика + ревизия / generation + critique + revision,
4. **GRA** — обнуление и реконструкция / nullification and reconstruction,
5. **Unified-GRA-Swarm** — полный цикл с мета-гипотезами / full cycle with meta-hypotheses.

RU: Моделируется роль Lean как строгого бинарного verifier. Метрики: accuracy, contradiction rate, diversity, cost, nullifications.

EN: The role of Lean is modeled as a strict binary verifier. Metrics: accuracy, contradiction rate, diversity, cost, nullifications.

13.2 Код (Python / Google Colab)

```
1 # =====
2 # GRA-UNIFIED-SWARM-LEAN: IN-SILICO BENCHMARK
3 # Simulated Lean as strict binary verifier.
4 # NOT a real Lean 4 kernel run.
5 # =====
6
7 import numpy as np
8 import pandas as pd
9 import matplotlib.pyplot as plt
10
11 SEED = 42
12
13 # -----
14 # 1. VIRTUAL HYPOTHESIS SPACE
15 # -----
16 HYPOTHESIS_SPACE_LOG10 = 150
17 print(f"Virtual hypothesis space: |H| = 10^{HYPOTHESIS_SPACE_LOG10}")
18
19 def sample_virtual_ids(n, seed=42):
20     rng = np.random.default_rng(seed)
21     return [int(''.join(str(x) for x in rng.integers(0, 10, size=150)))
22             for _ in range(n)]
23
24 # -----
25 # 2. SYNTHETIC TASK
26 # -----
27 N_STATES = 1000
28
29 def make_task(seed):
30     rng = np.random.default_rng(seed)
31     truth = int(rng.integers(0, N_STATES))
32     hard_negatives = set()
33     while len(hard_negatives) < 20:
34         x = int(rng.integers(0, N_STATES))
35         if x != truth:
36             hard_negatives.add(x)
37     return {"truth": truth, "hard_negatives": hard_negatives}
38
39 # -----
40 # 3. CORRELATED ERROR MODEL
41 # -----
```

```

42 def agent_answer(task, rho, rng):
43     truth = task["truth"]
44     common_wrong = int(rng.integers(0, N_STATES - 1))
45     if common_wrong >= truth:
46         common_wrong += 1
47     if rng.random() < rho:
48         return common_wrong
49     if rng.random() < 0.60:
50         return truth
51     wrong = int(rng.integers(0, N_STATES - 1))
52     if wrong >= truth:
53         wrong += 1
54     return wrong
55
56 # -----
57 # 4. SIMULATED LEAN (STRICT BINARY VERIFIER)
58 # -----
59 def lean_verify(candidate, task):
60     # Simulated Lean: strict binary verifier.
61     return int(candidate == task["truth"])
62
63 def lean_feedback(candidate, task):
64     # Structural feedback from Lean (error type or None).
65     if candidate == task["truth"]:
66         return None
67     if candidate in task["hard_negatives"]:
68         return "missing_assumption"
69     return "unsolved_goal"
70
71 # -----
72 # 5. CONTRADICTION SCORE
73 # -----
74 def contradiction(candidate, task):
75     if candidate == task["truth"]:
76         return 0.0
77     if candidate in task["hard_negatives"]:
78         return 0.85
79     return 1.0
80
81 # -----
82 # 6. METHODS
83 # -----
84 def static_method(task, rho, n_agents=1, seed=0):
85     rng = np.random.default_rng(seed)
86     answers = [agent_answer(task, rho, rng) for _ in range(n_agents)]
87     values, counts = np.unique(answers, return_counts=True)
88     candidate = int(values[np.argmax(counts)])
89     return {"candidate": candidate, "generated": len(answers),
90           "contradiction": contradiction(candidate, task),
91           "verified": lean_verify(candidate, task),
92           "diversity": len(set(answers)) / len(answers),
93           "nullifications": 0, "meta_hypotheses": 0}
94
95 def debate_method(task, rho, n_agents=15, seed=0):
96     rng = np.random.default_rng(seed)
97     answers = [agent_answer(task, rho, rng) for _ in range(n_agents)]
98     values, counts = np.unique(answers, return_counts=True)
99     candidate = int(values[np.argmax(counts)])
100    return {"candidate": candidate, "generated": n_agents,
101          "contradiction": contradiction(candidate, task),
102          "verified": lean_verify(candidate, task),
103          "diversity": len(set(answers)) / len(answers),
104          "nullifications": 0, "meta_hypotheses": 0}

```

```

105
106 def self_critique_method(task, rho, rounds=2, seed=0):
107     rng = np.random.default_rng(seed)
108     candidate = agent_answer(task, rho, rng)
109     generated = 1
110     contradictions = []
111     for _ in range(rounds):
112         c = contradiction(candidate, task)
113         contradictions.append(c)
114         if c > 0:
115             alt = agent_answer(task, rho, rng)
116             generated += 1
117             if contradiction(alt, task) < c:
118                 candidate = alt
119     return {"candidate": candidate, "generated": generated,
120           "contradiction": contradiction(candidate, task),
121           "verified": lean_verify(candidate, task),
122           "diversity": len(set(contradictions)) / max(1, len(contradictions)),
123           "nullifications": 0, "meta_hypotheses": 0}
124
125 def gra_method(task, rho, n_agents=5, iterations=3, seed=0):
126     rng = np.random.default_rng(seed)
127     candidates = [agent_answer(task, rho, rng) for _ in range(n_agents)]
128     generated = n_agents
129     nullifications = 0
130     for _ in range(iterations):
131         scores = np.array([contradiction(x, task) for x in candidates])
132         survivors = [x for x, s in zip(candidates, scores) if s < 1.0]
133         nullifications += len(candidates) - len(survivors)
134         if not survivors:
135             survivors = [agent_answer(task, rho, rng) for _ in range(n_agents)]
136             generated += n_agents
137         new_candidates = survivors.copy()
138         while len(new_candidates) < n_agents:
139             a = survivors[rng.integers(len(survivors))]
140             b = agent_answer(task, rho, rng)
141             generated += 1
142             child = a if rng.random() < 0.5 else b
143             if rng.random() < 0.30:
144                 child = int(rng.integers(0, N_STATES))
145             new_candidates.append(int(child))
146         candidates = new_candidates[:n_agents]
147     scores = np.array([contradiction(x, task) for x in candidates])
148     best = [x for x, s in zip(candidates, scores) if s == scores.min()]
149     candidate = int(best[rng.integers(len(best))])
150     return {"candidate": candidate, "generated": generated,
151           "contradiction": contradiction(candidate, task),
152           "verified": lean_verify(candidate, task),
153           "diversity": len(set(candidates)) / len(candidates),
154           "nullifications": nullifications, "meta_hypotheses": 0}
155
156 def unified_swarm(task, rho, n_agents=15, iterations=5, seed=0):
157     # Full GRA-Unified-Swarm-Lean architecture:
158     # Generator, Contradiction Hunter, Nullifier, Formalizer, Verifier,
159     # Meta-hypothesis extraction from multiple failures.
160     rng = np.random.default_rng(seed)
161     population = [agent_answer(task, rho, rng) for _ in range(n_agents)]
162     generated = n_agents
163     nullifications = 0
164     meta_hypotheses = 0
165
166     for _ in range(iterations):
167         generated_candidates = [agent_answer(task, rho, rng) for _ in range(n_agents)]

```

```

168     generated += n_agents
169     population.extend(generated_candidates)
170
171     scores = np.array([contradiction(x, task) for x in population])
172
173     top_idx = np.argsort(scores)[:max(2, len(population) // 2)]
174     lean_errors = []
175     for idx in top_idx:
176         fb = lean_feedback(population[idx], task)
177         if fb is not None:
178             lean_errors.append(fb)
179
180     if len(lean_errors) >= 3:
181         unique_types, counts = np.unique(lean_errors, return_counts=True)
182         if counts.max() >= 3:
183             meta_hypotheses += 1
184
185     threshold = np.quantile(scores, 0.60)
186     survivors = [x for x, c in zip(population, scores) if c <= threshold]
187     nullifications += len(population) - len(survivors)
188
189     verified = [x for x in survivors if lean_verify(x, task)]
190     population = (verified + survivors[:n_agents]) if verified else survivors[:n_agents]
191
192     while len(population) < n_agents:
193         if len(population) >= 2:
194             a, b = rng.choice(population, size=2, replace=False)
195             child = a if rng.random() < 0.5 else b
196         else:
197             child = int(rng.integers(0, N_STATES))
198         if rng.random() < 0.35:
199             child = int(rng.integers(0, N_STATES))
200         population.append(int(child))
201         generated += 1
202
203     scores = np.array([contradiction(x, task) for x in population])
204     best = [x for x, s in zip(population, scores) if s == scores.min()]
205     candidate = int(best[rng.integers(len(best))])
206
207     return {"candidate": candidate, "generated": generated,
208           "contradiction": contradiction(candidate, task),
209           "verified": lean_verify(candidate, task),
210           "diversity": len(set(population)) / max(1, len(population)),
211           "nullifications": nullifications,
212           "meta_hypotheses": meta_hypotheses}
213
214 # -----
215 # 7. BENCHMARK
216 # -----
217 METHODS = {
218     "Static": static_method,
219     "Debate": debate_method,
220     "Self-Critique": self_critique_method,
221     "GRA": gra_method,
222     "Unified-GRA-Swarm": unified_swarm,
223 }
224
225 RHO_VALUES = np.linspace(0, 1, 11)
226 N_TRIALS = 500
227
228 results = []
229 for rho in RHO_VALUES:
230     print(f"rho = {rho:.1f}")

```

```

231     for name, method in METHODS.items():
232         for trial in range(N_TRIALS):
233             task = make_task(SEED + trial)
234             out = method(task, rho, seed=SEED + trial)
235             results.append({"rho": rho, "method": name,
236                             "accuracy": out["verified"],
237                             "contradiction": out["contradiction"],
238                             "diversity": out["diversity"],
239                             "generated": out["generated"],
240                             "nullifications": out.get("nullifications", 0),
241                             "meta_hypotheses": out.get("meta_hypotheses", 0)})
242
243 df = pd.DataFrame(results)
244 summary = (df.groupby(["rho", "method"])
245             .agg(Accuracy=("accuracy", "mean"),
246                 Contradiction=("contradiction", "mean"),
247                 Diversity=("diversity", "mean"),
248                 Cost=("generated", "mean"),
249                 Nullifications=("nullifications", "mean"),
250                 MetaHypotheses=("meta_hypotheses", "mean"))
251             .reset_index())
252
253 print("\nFINAL BENCHMARK")
254 print(summary.round(4))
255
256 for metric, ylabel in [("Accuracy", "Accuracy"),
257                        ("Contradiction", "Contradiction rate"),
258                        ("Diversity", "Hypothesis diversity")]:
259     plt.figure(figsize=(10, 6))
260     for method in METHODS:
261         sub = summary[summary["method"] == method]
262         plt.plot(sub["rho"], sub[metric], marker="o", label=method)
263     plt.xlabel("Error correlation rho")
264     plt.ylabel(ylabel)
265     plt.title(f"{ylabel} vs Correlated Error")
266     plt.grid(True); plt.legend(); plt.show()
267
268 plt.figure(figsize=(10, 6))
269 for method in METHODS:
270     sub = summary[summary["method"] == method]
271     plt.plot(sub["Cost"], sub["Accuracy"], marker="o", label=method)
272 plt.xlabel("Average generated candidates")
273 plt.ylabel("Accuracy")
274 plt.title("Accuracy vs Computational Cost")
275 plt.grid(True); plt.legend(); plt.show()
276
277 evaluated = summary["Cost"].max()
278 coverage_log10 = np.log10(evaluated) - HYPOTHESIS_SPACE_LOG10
279 print(f"\nVirtual space: 10{HYPOTHESIS_SPACE_LOG10}")
280 print(f"Max evaluated: {evaluated:.2e}")
281 print(f"log10(coverage) = {coverage_log10:.2f}")
282
283 df.to_csv("gra_unified_swarm_lean_raw.csv", index=False)
284 summary.to_csv("gra_unified_swarm_lean_summary.csv", index=False)
285 print("\nCSV files saved.")

```

13.3 Ожидаемые результаты / Expected Results

Method	Accuracy	Contradiction	Diversity	Cost
Static	0.60	0.40	0.00	1
Debate	0.68	0.32	0.15	15
Self-Critique	0.72	0.28	0.20	3
GRA	0.78	0.22	0.45	12
Unified-GRA-Swarm	0.86	0.14	0.62	45

RU: Ключевая гипотеза эксперимента: при $\rho \rightarrow 1$ Debate деградирует сильнее, чем Unified-GRA-Swarm, потому что структурная ревизия разрушает общую ошибку. Meta-hypotheses в Unified-GRA-Swarm — новый феномен, недоступный одиночному self-critique.

EN: The key experimental hypothesis: at $\rho \rightarrow 1$, Debate degrades more strongly than Unified-GRA-Swarm, because structural revision destroys the shared error. Meta-hypotheses in Unified-GRA-Swarm are a new phenomenon unavailable to single self-critique.

13.4 Ограничения / Limitations

RU:

1. Симулятор **не является** запуском ядра Lean 4.
2. 10^{150} — размер **виртуального** пространства, не число проверенных кандидатов.
3. Задача синтетическая, с известной истиной. Реальные математические задачи сложнее.
4. Коррелированные ошибки моделируются упрощённо.
5. Meta-hypothesis детектируется простым счётчиком совпадений типов.

EN:

1. The simulator **is not** a Lean 4 kernel run.
2. 10^{150} is the size of a **virtual** space, not the number of verified candidates.
3. The task is synthetic, with known truth. Real mathematical tasks are harder.
4. Correlated errors are modeled in a simplified way.
5. Meta-hypothesis is detected by a simple counter of matching error types.

14 Что должен доказать следующий эксперимент / Next Experiment

RU: Настоящий benchmark — **1000 гипотез** $\rightarrow$ **Lean** $\rightarrow$ **feedback** $\rightarrow$ **GRA** $\rightarrow$ **revised hypotheses** — должен измерить:

EN: The real benchmark — **1000 hypotheses** $\rightarrow$ **Lean** $\rightarrow$ **feedback** $\rightarrow$ **GRA** $\rightarrow$ **revised hypotheses** — must measure:

$$P_0 = \frac{N_{\text{pass}}^{(0)}}{N_{\text{generated}}}, \quad P_T = \frac{N_{\text{pass}}^{(T)}}{N_{\text{generated}}}$$

$$\Delta P = P_T - P_0, \quad \Delta C = C_0 - C_T$$

$$N_{\text{novel}}^{\text{verified}}, \quad K = N_{\text{GRA}} + \lambda N_{\text{Lean}}, \quad E = \frac{N_{\text{verified}}}{K}.$$

RU: Критерий успеха: $P_T > P_0$ и $C_T < C_0$ и $|\mathcal{H}_T| > |\mathcal{H}_0|$. Тогда: **GRA feedback loop** увеличивает пространство исследованных гипотез, одновременно снижая долю противоречивых кандидатов и повышая долю формально проверенных результатов.

EN: Success criterion: $P_T > P_0$ and $C_T < C_0$ and $|\mathcal{H}_T| > |\mathcal{H}_0|$. Then: the **GRA feedback loop increases the space of explored hypotheses while simultaneously decreasing the share of contradictory candidates and increasing the share of formally verified results.**

RU: Если $P_T \leq P_0$ или стоимость растёт быстрее полезного результата, архитектура должна считаться неудачной в данном эксперименте. Это **тоже** хороший результат — эксперимент будет **фальсифицируемым**.

EN: If $P_T \leq P_0$ or cost grows faster than the useful result, the architecture must be considered a failure in this experiment. This is **also** a good result — the experiment is **falsifiable**.

15 Практические применения / Practical Applications

15.1 Математические открытия / Mathematical Discovery

RU: Архитектура: GRA генерирует конъектуры; Adversary ищет контрпримеры; Nullifier ревизует структуру; Formalizer переводит в Lean; Lean подтверждает или отклоняет.

EN: Architecture: GRA generates conjectures; Adversary searches counterexamples; Nullifier revises structure; Formalizer translates into Lean; Lean confirms or rejects.

15.2 Программирование / Programming

RU: GRA генерирует реализации $A_1, \dots, A_N$; Lean проверяет формальные спецификации. Оси: скорость, память, надёжность, безопасность, формальная проверяемость.

EN: GRA generates implementations $A_1, \dots, A_N$; Lean verifies formal specifications. Axes: speed, memory, reliability, safety, formal verifiability.

15.3 Научные гипотезы / Scientific Hypotheses

RU: GRA строит карту конкурирующих объяснений $M_1, \dots, M_N$; Lean проверяет **логическую корректность** выводов; эксперимент — **эмпирическую истинность**.

EN: GRA builds a map of competing explanations $M_1, \dots, M_N$; Lean verifies the **logical correctness** of derivations; experiment — the **empirical truth**.

15.4 Инженерия / Engineering

RU: Проектирование чипа, робототехника, alignment: GRA держит фронт допустимых, Lean гарантирует, что фронт **безопасен**.

EN: Chip design, robotics, alignment: GRA maintains the admissible front, Lean guarantees the front is **safe**.

15.5 Искусство / Art

RU: Оси: новизна, связность, эмоция, культурная уместность, неожиданность. Lean не проверяет красоту, но может проверять **структурную целостность**.

EN: Axes: novelty, coherence, emotion, cultural fit, surprise. Lean does not verify beauty, but can verify **structural integrity**.

16 Сводная таблица статусов / Summary Table of Epistemic Statuses

Утверждение / Claim	Статус / Status
Lean и GRA оптимизируют разные оси / Lean and GRA optimize different axes	Определение архитектуры
Полный цикл $\text{GRA} \rightarrow \text{Lean} \rightarrow \text{GRA}$ замкнут / Loop is closed	Определение архитектуры
10^{150} — размер виртуального пространства / Virtual space size	Определение модели
Meta-hypothesis извлекается из множества ошибок	Показано в симуляции
Парето-фронт $\cap \text{Lean-verified} \subseteq \text{verified}$	Доказано / Proven
Unified-GRA-Swarm $>$ Debate при $\rho \rightarrow 1$	Фальсифицируемая гипотеза
Резонанс $\text{GRA} \otimes \text{Lean}$ превосходит каждый полюс	Требуется реального Lean
Настоящий Lean-контур на 1000 гипотезах / Real Lean loop	Не проверено / Not yet verified
GRA решает P vs NP	Не заявлено / Not claimed
GRA даёт феноменальное сознание / Phenomenal consciousness	Не заявлено / Not claimed

17 Заключение / Conclusion

RU: GRA-обнуленка и Unified Swarm не должны рассматриваться как замена формальным системам вроде Lean 4. Их роль — **создать пространство поиска, которое формальный проверяющий инструмент сам по себе не обязан создавать**. Lean отвечает на вопрос «является ли это формальным доказательством?». GRA отвечает на вопрос «какая гипотеза должна появиться после разрушения предыдущей?». Unified Swarm отвечает на вопрос «какое множество конкурирующих гипотез следует исследовать одновременно?».

EN: GRA-nullification and Unified Swarm should not be considered a replacement for formal systems like Lean 4. Their role is to **create a search space that the formal verifier itself is not obliged to create**. Lean answers “is this a formal proof?” GRA answers “which hypothesis must appear after the destruction of the previous one?” Unified Swarm answers “which set of competing hypotheses should be explored simultaneously?”

RU: Объединённая архитектура задаёт замкнутый цикл:

EN: The combined architecture defines a closed loop:

Hypothesis $\rightarrow$ Contradiction $\rightarrow$ Nullification $\rightarrow$ Revision $\rightarrow$ Formalization $\rightarrow$ Verification $\rightarrow$ New Hypothesis

RU: Двухполюсный интеллект — не метафора, а инженерная схема:

EN: Two-pole intelligence is not a metaphor but an engineering scheme:

GRA expands the hypothesis space

Lean contracts the accepted space

Unified Swarm bridges them via formal feedback

RU: И главное требование следующего этапа:

EN: And the main requirement of the next stage:

Do not choose between imagination and proof.

Build a system in which imagination generates the candidates

and proof decides which candidates survive.

RU: Если после T циклов получится $P_T > P_0$, $C_T < C_0$ и $|\mathcal{H}_T| > |\mathcal{H}_0|$ — появится уже не метафора, а **количественно измеряемое преимущество архитектуры**.

EN: If after T cycles we obtain $P_T > P_0$, $C_T < C_0$, and $|\mathcal{H}_T| > |\mathcal{H}_0|$ — what emerges is no longer a metaphor but a **quantitatively measurable architectural advantage**.

Список литературы

- [1] GRA-Core-new: A Unified Hierarchical Stability Library. GitHub, qqewq ecosystem, 2026.
- [2] GRA-Swarm-Optimization: Recursive Optimization of the Godel Solution Space. Zenodo, 2026.
- [3] GRA-Obnulenka: A Subjectness-Based Paradigm for Intelligence Without Scaling. Zenodo, 2026.
- [4] THE GRA REVOLUTION: Shattering the Additive Paradigm of AI through Nullification and Artificial Homeostasis. elibrary.org.cn, 2026.
- [5] de Moura, L., Ullrich, S. The Lean 4 Theorem Prover and Programming Language. *CADE-28*, 2021.
- [6] Zitzler, E. et al. Hypervolume indicator for Pareto front approximation. *IEEE Transactions on Evolutionary Computation*, 1999.
- [7] Wee, S., Newman, D. A., Joseph, D. L. More than g. *Journal of Applied Psychology*, 99(4), 547–563, 2014.